Unit-1 - Question 1

Write about the following terms.

Write about the following terms: i) Keywords ii) Identifiers iii) Data types iv) Variables v) Comments vi) Quotations.

i) Keywords

- Keywords are reserved words in Python.
- They have special meaning and cannot be used as identifiers (names).
- Python is case-sensitive.
- Examples of keywords:

```
False, None, True, and, as, assert, break, class, continue, def, del,
elif, else, except, finally, for, from, global, if, import, in, is, lambda,
nonlocal, not, or, pass, raise, return, try, while, with, yield
```

- Example:
  - `if, for, while, return` → valid keywords
  - `If, True, for1` → not keywords (If - wrong case, true - not keyword, for1 - has digit)

ii) Identifiers

- Identifiers are names given to variables, functions, classes, etc.
- Rules:
  - → Must start with a letter (a-z, A-Z) or underscore (_).
  - → Can contain letters, digits (0-9) and underscore (_).
  - → Cannot start with a digit.
  - → Cannot be a keyword.
  - → Case-sensitive (age, Age and AGE are different).
- Examples:

Valid identifiers	Invalid identifiers
age, student_name, _temp, total2, PI, count_1, myVar	2age (starts with digit), student-name (hyphen not allowed), class (keyword), for (keyword), total$ (special char)

iii) Data types

- Data types define the type of value a variable can store.
- Python has several built-in data types. Common ones:
  - → int : Stores integer numbers. Example: `x = 10`
  - → float : Stores decimal numbers. Example: `pi = 3.14`
  - → str : Stores text (string). Example: `name = "Riya"`
  - → bool : Stores boolean values True or False. Example: `flag = True`
- (Python also has other types like complex, list, tuple, set, dict, etc.)

iv) Variables

- A variable is a named memory location used to store data.
- Created when a value is assigned to it using '=' operator.
- Does not require declaration of type; Python is dynamically typed.
- Example:

```
x = 25              # x stores integer 25
price = 99.75       # price stores float value
name = "Amit"       # name stores string
is_passed = True    # is_passed stores boolean value
```

v) Comments

- Comments are used to write explanations in code for readability.
- Python ignores comments during execution.
- Types:
  1. Single-line comment : starts with '#'.

```
# This is a single-line comment
```

  2. Multi-line comment : using triple quotes ('''...''').

```
'''
This is a
multi-line comment
'''
```

- Example:

```
x = 10      # initialize x
y = 5       # initialize y
# Calculate sum
s = x + y   # s stores sum
'''
This program adds
two numbers.
'''
```

vi) Quotations

- Quotations are used to represent string literals in Python.
- Python allows both single quotes (') and double quotes (").
- Triple quotes (''' or """) are used for multi-line strings.
- Examples:

```
a = 'Hello'                 # using single quotes
b = "Python"                # using double quotes
c = "Python's power"        # single quote inside double quotes
d = 'He said, "Hi!"'        # double quote inside single quotes
e = """This is
  a multi-line string."""   # multi-line string
```

Unit-1 – Question 2

Various operators in Python

Question 2 : List and explain various operators in Python with suitable examples.

Operators are special symbols used to perform operations on operands (values / variables). Python provides the following main types of operators.

1) **Arithmetic operators** : Used to perform mathematical operations.

Operator	Meaning	Example (a=10, b=3)	Result
+	Addition	a + b	13
-	Subtraction	a - b	7
*	Multiplication	a * b	30
/	Division (float)	a / b	3.333...
//	Floor Division (quotient)	a // b	3
%	Modulus (remainder)	a % b	1
**	Exponentiation (power)	a ** b	1000

2) **Assignment operators** : Used to assign values to variables.

Operator	Meaning	Example	Same as
=	Assign	a = 10	a = 10
+=	Add and assign	a += 3	a = a + 3
-=	Subtract and assign	a -= 2	a = a - 2
*=	Multiply and assign	a *= 4	a = a * 4
/=	Divide and assign	a /= 5	a = a / 5
//=	Floor divide and assign	a //= 3	a = a // 3
%=	Modulus and assign	a %= 3	a = a % 3
**=	Exponentiate and assign	a **= 2	a = a ** 2

3) **Comparison or Relational operators** : Used to compare two values.

Operator	Meaning	Example (a=10, b=3)	Result
==	Equal to	a == b	False
!=	Not equal to	a != b	True
>	Greater than	a > b	True
<	Less than	a < b	False
>=	Greater than or equal to	a >= b	True
<=	Less than or equal to	a <= b	False

4) **Logical operators** : Used to combine conditional statements.

Operator	Meaning	Example	Result
and	True if both conditions are True	(a > 5 and b < 5)	True
or	True if any one condition is True	(a < 5 or b > 5)	True
not	Reverse the result	not(a > b)	False

5) **Bitwise operators** : Operate on bits of integers.

Operator	Meaning	Example (a=10, b=3)	Result
&	Bitwise AND	a & b	2 (10 & 3 = 2)
|	Bitwise OR	a | b	11 (10 | 3 = 11)
^	Bitwise XOR	a ^ b	9 (10 ^ 3 = 9)
~	Bitwise NOT (Ones' complement)	~a	-11 (because ~10 = -11)
<<	Left shift	a << 1	20 (10 << 1 = 20)
>>	Right shift	a >> 1	5 (10 >> 1 = 5)

6) **Membership operators** : Used to test membership in a sequence.

Operator	Meaning	Example	Result
in	True if value is present	'a' in 'apple'	True
not in	True if value is not present	'z' not in 'apple'	True

7) **Identity operators** : Used to compare memory locations (identity).

Operator	Meaning	Example	Result
is	True if both are same object	a is b	False (different objects)
is not	True if both are not same object	a is not b	True

Conclusion : Operators are symbols used to perform operations on operands.

Unit-1 - Question 3

List and explain the features of Python.

- **Easy to learn** : Python has clear and simple syntax which is easy for beginners to read, write and understand.
- **Simple syntax** : The code looks similar to English and uses indentation, which makes programs neat and readable.
- **Interpreted language** : Python programs are executed line-by-line by the interpreter. There is no separate compilation step.
- **High-level language** : Python abstracts complex machine-level details and provides easy-to-use built-in data types and functions.
- **Object-oriented** : Python supports classes and objects. It supports important OOP concepts like encapsulation, inheritance and polymorphism.
- **Portable** : Python programs can run on different operating systems such as Windows, Linux, macOS, etc., without changes.
- **Free and open source** : Python is freely available and open source. Anyone can use, modify and distribute it.
- **Large standard library** : Python has a rich collection of modules and packages for various tasks like file handling, math, networking, GUI, databases, etc.
- **Dynamic typing** : Variables are not required to declare their data type. The type is decided automatically at runtime.

Ex:
```
x = 10
x = "Hello"
x = 3.14
```

- **Extensible** : Python can be extended by adding new modules written in C, C++ or other languages.
- **Interactive** : Python provides an interactive shell (IDLE) where we can write and execute code immediately. Useful for testing.
- **Supports GUI** : Python has many libraries (like tkinter, PyQt) to create graphical user interface (GUI) applications.
- **Supports multiple programming paradigms** : Python supports procedural, object-oriented and functioming.
- **Wide range of applications** : Python is used in many fields such as web development, artificial intelligence, machine learning, data science, automation, scripting, networking, games, etc.

Note / Example :

1) Easy and readable syntax

```
print("Hello, Python!")        # Simple and clear
```

2) No need to declare datatype

```
a = 25                         # Integer
a = 25.5                       # Float (type changes automatically)
```

Unit-1 - Question 4

Various if statements in Python

Question 4: With syntax and program explain various if statements (if, if-else, if-elif-else).

In Python, decision making is done using if statements. They allow the program to execute a block of code only when a condition is True.

① if statement

Meaning: Executes a block of code only if the given condition is True. If the condition is False, the block is skipped and the program moves to the next statement.

Program:

```
n = int(input("Enter a number: "))
if n > 0:
    print("The number is positive.")
```

Syntax:

```
if condition:
    # block of code
```

Example:

Input: 7
Output: The number is positive.

② if-else statement

Meaning: Executes one block if the condition is True and another block if the condition is False.

Program:

```
n = int(input("Enter a number: "))
if n % 2 == 0:
    print("The number is even.")
else:
    print("The number is odd.")
```

Syntax:

```
if condition:
    # block of code (True part)
else:
    # block of code (False part)
```

Example:

Input: 10
Output: The number is even.

(or)

Input: 7
Output: The number is odd.

③ if-elif-else statement

Meaning: Used to test multiple conditions. The program checks the conditions from top to bottom and executes the first True block. If none of the conditions is True, the else block is executed.

Program:

```
marks = int(input("Enter marks (0-100): "))
if marks >= 90:
    grade = "A"
elif marks >= 75:
    graks = "B"
elif marks >= 50:
    grade = "C"
else:
    grade = "D"
print("Grade:", grade)
```

Syntax:

```
if condition1:
    # block of code
elif condition2:
    # block of code
elif condition3:
    # block of code
    ...
else:
    # block of code
```

Example:

Input: 82
Output: Grade: B

(or)

Input: 45
Output: Grade: D

Important Notes:

- Indentation: Indentation is mandatory in Python. The block of code inside if, else, or elif must be indented (usually 4 spaces).
- Conditional Execution: Only the block whose condition is True is executed.
- Evaluation Order: In if-elif-else, conditions are checked from top to bottom. As soon as one condition is True, its block runs and the rest are skipped.
- Use proper conditions and comparison operators (==, !=, >, <, >=, <=) to build decisions.

Unit-1 – Question 5

for statement with sequence and range() function

Question 5: Explain for statement with sequence and range() function along with example.

1. for statement (for loop)

* The for loop is used to iterate over a sequence (a collection of items).
* It executes a block of statements once for each item in the sequence.
* It is especially useful when the number of iterations is known or when we want to process every item in a collection.

Syntax:

```
for variable in sequence:
    statements
```

→ variable takes each value from the sequence one by one.

2. Sequence

* A sequence is an ordered collection of items.
* Python has several built-in sequences.
* Common sequences:
  - i. String : `'hello'` , `"Python"`
  - ii. List : `[10, 20, 30]` , `['a', 'b', 'c']`
  - iii. Tuple : `(1, 2, 3)` , `('x', 'y', 'z')`

Note: All these can be used in a for loop because each item can be accessed one by one.

3. range() function

* range() generates a sequence of numbers.
* It is commonly used with for loop.
* range() does not include the stop value.

i. `range(stop)` : Generates numbers from 0 to stop-1 (default start is 0, step is 1)
   Example: range(5) → 0, 1, 2, 3, 4

ii. `range(start, stop)` : Generates numbers from start to stop-1 (default step is 1)
   Example: range(2, 6) → 2, 3, 4, 5

iii. `range(start, stop, step)` : Generates numbers from start to stop-1 with given step.
   Example: range(1, 10, 2) → 1, 3, 5, 7, 9

4. Simple Programs

(i) Print numbers 1 to 5

```
for i in range(1, 6):
    print(i)
```

Output:
```
1
2
3
4
5
```

(ii) Print characters of a string

```
s = "Python"
for ch in s:
    print(ch)
```

Output:
```
P
y
t
h
o
n
```

(iii) Print even numbers up to 10

```
for i in range(2, 11, 2):
    print(i)
```

Output:
```
2
4
6
8
10
```

Summary: for loop is used to iterate over sequences. range() helps to generate number sequences easily. It makes programs short, simple and easy to read.

Unit-1 - Question 6

Explain loop control statements (break, continue, pass) with necessary programs.

Loop control statements are used to alter the normal flow of loops (for, while) to make the program more efficient and logical.

1. break

- Meaning : Terminates the loop immediately and transfers the control to the statement just after the loop.
- Use : When a required condition is met and we want to stop the loop.
- Syntax idea :
```
if condition:
    break      # exit the nearest loop
```
- Example : Stop searching when a number is found.

```
num = int(input("Enter a number to find: "))
found = False
for i in range(1, 21):
    if i == num:
        print("Number found:", i)
        found = True
        break        # exit the loop

if not found:
    print("Number not found in 1 to 20")
```

How it works:
When i equals num, break stops the loop immediately.

2. continue

- Meaning : Skips the remaining statements inside the loop for the current iteration and moves to the next iteration.
- Use : When we want to skip certain values/iterations.
- Syntax idea :
```
if condition:
    continue      # skip to next iteration
```
- Example : Print numbers from 1 to 10, skipping 5.

```
for i in range(1, 11):
    if i == 5:
        continue      # skip printing 5
    print(i, end=" ")
```

Output: 1 2 3 4 6 7 8 9 10

How it works:
When i is 5, continue skips the print statement and jumps to the next value of i.

3. pass

- Meaning : Does nothing. It is a null statement.
- Use : When a block is required syntactically but we don't want to write any code yet. Acts as a placeholder.
- Syntax idea : pass
- Example : Placeholder in loops / functions / classes.

(a) In a loop

```
for i in range(3):
    pass    # to be written later
```

(b) In a function

```
def add(a, b):
    pass    # implementation later
```

Comparison:

Statement	Effect on loop	Next control goes to	Typical use
break	Terminates the loop completely	First statement after the loop	Stop the loop when condition met
continue	Skips current iteration	Next iteration of the loop	Skip unwanted values/iterations
pass	No effect (does nothing)	Next statement in the block	Placeholder/ empty block

Note: All three statements can be used in both for and while loops.
break and continue do not work outside loops; pass can be used anywhere.

Unit-1 - Question 7

Working of else with loops

Question 7: Explain the working of else with loops with examples.

In Python, a loop can have an optional **else** block.
The else part is executed only if the loop terminates **normally** (i.e., the condition becomes False or the iterable is exhausted).
If the loop is terminated by a **break** statement, the else block is **skipped**.

1) else with for loop

Syntax:

```
for item in sequence:
    # loop body
else:
    # else block
```

- The else block runs after the for loop completes all iterations without hitting break.
- If break is executed inside the loop, the else block is not executed.

Example: Search an element in a list

```
nums = [10, 20, 30, 40, 50]
key = 30
for x in nums:
    if x == key:
        print ("Found", key)
        break
else:
    print ("Not found")        # runs only if loop
                               # completes without break
```

Output:
```
Found 30
```

```
key = 60
for x in nums:
    if x == key:
        print ("Found", key)
        break
else:
    print ("Not found")        # else executes
```

Output:
```
Not found
```

2) else with while loop

Syntax:

```
while condition:
    # loop body
else:
    # else block
```

- The else block runs after the while loop condition becomes False.
- If break is executed, else block is skipped.

Example: Count from 1 to n using while loop

```
n = 5
i = 1
while i <= n:
    print (i, end=" ")
    i = i + 1
else:
    print ("\nCounting completed!")
                               # runs because
                               # no break occurred
```

Output:
```
1 2 3 4 5
Counting completed!
```

```
n = 5
i = 1
while i <= n:
    if i == 3:
        break
    print (i, end=" ")
    i = i + 1
else:
    print ("\nThis will not print!")   # else skipped
                                       # due to break
```

Output:
```
1 2
```

Important Notes:

1. else with loops is optional but useful.
2. It helps in writing cleaner code, especially for search and validation tasks.
3. Use break to exit early when the desired condition is met; otherwise else will run after normal completion.

Comparison

Feature	for loop with else	while loop with else
When else runs	After all items are iterated (no break)	When condition becomes False (no break)
If break occurs	else is skipped	else is skipped
Common use	Search in a list, check presence	Counting, input validation, retries

Remember: else with loops executes only on **normal completion**, not on break.

Unit-1 – Question 8

Nested loop with examples (pattern programs)

Question 8: Explain the concept of nested loop with examples (pattern programs).

1. What is Nested loop?

→ A nested loop is a loop inside another loop.
→ The inner loop executes completely for each iteration of the outer loop.
→ Used to solve problems that have rows & columns or repeated steps inside repeated steps.

2. Syntax:

```
for outer_loop_variable in range(outer_start, outer_end, outer_step):
    statements (outer loop)
    for inner_loop_variable in range(inner_start, inner_end, inner_step):
        statements (inner loop)
        more statements (inner loop)
more statements (outer loop)
```

3. Working:

→ Outer loop runs one time.
→ Then inner loop runs completely.
→ After inner loop finishes, control returns to outer loop for its next iteration.
→ This continues until outer loop finishes.

4. Examples:

(a) 3 x 3 Star Pattern

Code:

```
for i in range(3):
    for j in range(3):
        print('*', end=' ')
    print()  # new line after each row
```

Output:

```
* * *
* * *
* * *
```

(b) Number Pattern (1 to 3)

Code:

```
for i in range(1, 4):
    for j in range(1, 4):
        print(j, end=' ')
    print()
```

Output:

```
1 2 3
1 2 3
1 2 3
```

(c) Right Triangle Star Pattern

Code:

```
for i in range(1, 4):
    for j in range(i):
        print('*', end=' ')
    print()
```

Output:

```
*
* *
* * *
```

(d) Multiplication Table (1 to 5)

Code:

```
for i in range(1, 6):
    for j in range(1, 6):
        print(i*j, end='\t')
    print()
```

Output:

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25

5. Uses / Applications of Nested Loops:

- To solve problems that involve two-dimensional data (rows & columns).
- To print patterns, tables, matrices.
- In searching & sorting two-dimensional lists.
- In games (grids), graphics, and simulations.

Note: The inner loop always completes all its iterations before the outer loop moves to the next iteration.

Unit-1 - Question 9

Top-down and Bottom-up approaches

Question 9: Explain top-down and bottom-up approaches.

1. Top-down approach (Step-down approach)

- Top-down approach is a method of problem solving or program design in which the solution is developed from the overall problem to the details.
- The big problem is divided into smaller sub-problems (modules). Each sub-problem is further divided until simple tasks are obtained.
- It follows a hierarchical structure.

Steps:

① Understand and define the complete problem.
② Break the problem into major sub-problems (modules).
③ Further decompose each sub-problem into smaller sub-problems.
④ Continue until each sub-problem is simple enough to code.
⑤ Implement (code) the lowest-level modules.
⑥ Integrate all modules and test the complete system.

Advantages:

- Easy to understand the overall problem.
- Clear structure and planning.
- Large and complex problems can be handled easily.
- Each module can be developed by a different member (good for team work).
- Debugging is easier at higher level.

Example (Real-life / Program):

Building a house or developing a 'Library Management System'. First decide main parts - Login, Book management, Issue/Return, Fine calculation, Reports, etc. Then design each part in details.

2. Bottom-up approach (Step-up approach)

- Bottom-up approach is a method in which the solution is developed from the details or basic components to the overall system.
- First develop small, simple modules (building blocks).
- Then combine these modules to form larger sub-systems and finally the complete system.

Steps:

① Identify the smallest components or tasks required.
② Develop and test each small module.
③ Combine these modules to form larger modules.
④ Continue combining until the complete system is built.
⑤ Finally, test and integrate the entire system.

Advantages:

- Reusable components can be created.
- Each module can be tested individually.
- Problems in a module do not affect other modules.
- Good for component-based development.

Example (Real-life / Program):

Assembling a car or developing a 'Calculator' program. First create basic functions - add(), sub(), mul(), div(). Then use them to build the calculator program.

3. Difference between Top-down and Bottom-up approaches

Aspect	Top-down approach	Bottom-up approach
Direction	From general to specific.	From specific to general.
Start	Start from the overall problem.	Start from small components.
Design	Divide the problem into smaller parts.	Combine small parts into larger system.
Focus	Overall structure first.	Details and components first.
Modules	High-level modules are developed first.	Low-level modules are developed first.
Integration	Done at the end (after coding).	Done gradually (step by step).
Testing	Testing is done after the system is built.	Each module is tested before integration.
Suitability	Suitable for large and complex problems.	Suitable when reusable components exist.
Example	Designing a university management system.	Building a library using ready-made furniture pieces.

Conclusion: Both approaches are useful. Top-down is better for analysis and planning, while Bottom-up is better for reusability and reliability.

Unit-1 - Question 10

Basic Python programs

Q10. Programs (even or odd, factorial, Fibonacci, multiplication table, reverse of a number, palindrome, sum of individual digits, etc.).

1. Even or Odd

Check whether a given number is even or odd.

```
n = int(input("Enter a number:"))
if n % 2 == 0:
    print(n, "is Even")
else:
    print(n, "is Odd")
```

Example:

Input: 7
Output: 7 is Odd

2. Factorial of a Number

Find factorial of a non-negative integer.

```
n = int(input("Enter a number:"))
fact = 1
if n < 0:
    print("Factorial is not defined for negative numbers")
else:
    for i in range(1, n + 1):
        fact *= i
    print("Factorial =", fact)
```

Example:

Input: 5
Output: Factorial = 120

3. Fibonacci Series

Print first n terms of Fibonacci series.

```
n = int(input("Enter n:"))
a, b = 0, 1
print("Fibonacci series:")
for i in range(n):
    print(a, end=" ")
    a, b = b, a + b
```

Example:

Input: 6
Output: 0 1 1 2 3 5

4. Multiplication Table

Print multiplication table of a number.

```
n = int(input("Enter a number:"))
print("Multiplication table of", n)
for i in range(1, 11):
    print(n, "x", i, "=", n * i)
```

Example:

Input: 4
Output:

```
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
  ...
4 x 10 = 40
```

5. Reverse of a Number

Reverse the digits of a number.

```
n = int(input("Enter a number:"))
rev = 0
while n > 0:
    digit = n % 10
    rev = rev * 10 + digit
    n = n // 10
print("Reverse =", rev)
```

Example:
Input: 12345
Output: Reverse = 54321

6. Palindrome Number

Check whether a number is palindrome.

```
n = int(input("Enter a number:"))
temp, rev = n, 0
while n > 0:
    digit = n % 10
    rev = rev * 10 + digit
    n = n // 10
if temp == rev:
    print(temp, "is Palindrome")
else:
    print(temp, "is not Palindrome")
```

Example:
Input: 121
Output: 121 is Palindrome

7. Sum of Individual Digits

Find sum of digits of a number.

```
n = int(input("Enter a number:"))
s = 0
while n > 0:
    s += n % 10
    n = n // 10
print("Sum of digits =", s)
```

Example:
Input: 4567
Output: Sum of digits = 22
(4 + 5 + 6 + 7 = 22)

Note: These programs work for positive integers.